

Given this, how do I implement the normal formula with this?

lab0_exercises.ipynb:32-39

Analyzed normal equation implementation in NumPy

Looking at your code, there's a syntax issue. The `^` operator in Python is **bitwise XOR**, not matrix operations. For the normal equation $\theta = (X^T X)^{-1} X^T y$, you need to use:

- `@` for matrix multiplication
- `.T` for transpose
- `np.linalg.inv()` for matrix inverse

Here's the correct implementation:

```
# Method 1: Using np.linalg.inv()
theta_hat = np.linalg.inv(X.T @ X) @ X.T @ y

# Method 2: Using np.linalg.solve() (more numerically stable)
theta_hat = np.linalg.solve(X.T @ X, X.T @ y)
```

Breakdown:

- `X.T @ X` — `X` transpose times `X` (produces a 3x3 matrix)
- `np.linalg.inv(X.T @ X)` — inverse of that matrix
- `@ X.T @ y` — multiply by `X` transpose times `y`

Why Method 2 is better: `np.linalg.solve()` is more numerically stable than computing the inverse explicitly.

After computing `theta_hat`, you can compare it with `true_theta`:

```
print("Estimated theta:", theta_hat)
print("True theta:", true_theta)
print("Difference:", np.abs(theta_hat - true_theta))
```

📄 👍 💬 ...

Claude Haiku 4.5 • 1.2 credits

Copied this whole snippet from an assignment which was posted on GitHub, what could be wrong with the syntax?

lab0_exercises.ipynb:13-22

Analyzed syntax errors in provided code snippet

There are a few syntax errors in this snippet:

1. **Wrong brackets for `np.random.choice()`** — It uses `[]` instead of `()`:

```
np.random.choice['CS', 'Math', ...] ❌ Wrong  
np.random.choice([...], n_students) ✅ Correct
```

2. **Malformed list** — The choices list is broken across lines incorrectly. It should be:

```
['CS', 'Math', 'Physics', 'Biology']
```

Here's the corrected code:

```
data = {  
    'student_id': range(1000, 1000 + n_students),  
    'major': np.random.choice(['CS', 'Math', 'Physics', 'Biology'], n_students),  
    'year': np.random.choice([1, 2, 3, 4], n_students),  
    'exam_score': np.random.normal(75, 10, n_students).clip(0, 100),  
    'assignments_completed': np.random.randint(0, 11, n_students),  
    'hours_studied': np.random.normal(15, 5, n_students).clip(1, 40)  
}
```

Key fixes:

- Changed `np.random.choice[...]` to `np.random.choice([...])`
- Moved the list of majors onto one line to avoid line-break confusion
- Kept the proper argument order: choices first, then `n_students`

This looks like a copy-paste issue from GitHub where the formatting got corrupted. Always double-check when copying code snippets!

📄 👍 🗨️ ...

Claude Haiku 4.5 • 1.6 credits

Now I need you to check all the cells in this notebook to see if I have done all what the TODO comments ask

 lab0_exercises.ipynb

Analyzing notebook content

Checking the notebook for all `TODO` items and whether each is implemented.

Searched for `TODO` in lab0_exercises.ipynb

Reviewing notebook cells around every `TODO` section to verify actual completion and spot bugs.

Reviewed lab0_exercises.ipynb and evaluated tasks

Notebook TODO check

I reviewed all TODOs in `Quainoo_Albert_Lab0/lab0_exercises.ipynb`.

✅ Correctly implemented

- Exercise 4 Task 1: border matrix
- Exercise 4 Task 3: normal equation solved correctly
- Exercise 5 Task 1: missing values counted
- Exercise 5 Task 1: missing values filled for `exam_score` and `hours_studied`
- Exercise 5 Task 2: average exam score by major
- Exercise 5 Task 2: correlation calculation
- Exercise 5 Task 3: group stats by major/year
- Exercise 5 Task 3: top 5 students with ties
- Exercise 5 Task 3: pivot table
- Exercise 6: all visualizations
- Exercise 7: CLV, age groups, age-group stats, top 10%, and plots
- Exercise 7: analysis paragraph exists

⚠ Issues found

1. Exercise 4 Task 2 — not truly column-wise normalization

- Current code:

```
mean = random_data.mean()
std = random_data.std()
standardised = (random_data - mean) / std
```

- This normalizes using the global mean/std of the whole array, not per column.
- Fix:

```
mean = random_data.mean(axis=0)
std = random_data.std(axis=0)
standardised = (random_data - mean) / std
```

2. Exercise 5 Task 1 — `df.describe` is not called

- Current code:

```
print(df.describe)
```

- It should be:

```
print(df.describe())
```

3. Exercise 5 Task 2 — performance label bug

- Current code:

```
elif score >= 80:
    'Good'
```

- That branch does not return 'Good', so scores 80–89 become `None`.
- Fix:

```
elif score >= 80:
    return 'Good'
```

4. Exercise 5 Task 2 — best major is computed but not printed

- You calculate:

```
best_major = avg_by_major.idxmax()
best_score = avg_by_major.max()
```

Summary

Most TODOs are implemented, but there are three real issues:

- normalization should be column-wise,
- `df.describe()` needs parentheses,
- `performance_label` must return `'Good'`.

When working in a jupyter notebook, how I'm I supposed to add imports, when I use different cells?

In a Jupyter Notebook, imports are usually placed in the first code cell of the notebook.

For example:

```
<> Python  
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Once you run that cell, those imports are available to the other cells **below and above it**, as long as the notebook kernel is still running.

So you can do this:

```
<> Python  
# Cell 1
import pandas as pd
```

```
<> Python  
# Cell 2
data = pd.DataFrame({
    "Name": ["Ama", "Kofi"],
    "Score": [85, 90]
})

data
```

That works because `pd` was already defined when you ran Cell 1.

The key thing is: Jupyter remembers what you have run, not just what is written above.

So this can also work:



lab0_exercises.ipynb
File

Check if I have done everything in this notebook (as per the TODOs)

I'll inspect the notebook structure, find every TODO, and compare each one against the code that was actually filled in. Then I'll flag anything incomplete or likely incorrect.

Thought for 52s >

You have **attempted every TODO** in the notebook, but I would **not submit it yet** because two visualization sections currently fail when the notebook is run from top to bottom.

Overall verdict

Mostly complete, but not fully safe to submit until you fix the Seaborn barplot issue.

The main issue is this error:

Python



Run

ValueError: Invalid RGBA argument: None

It happens in:

1. Exercise 6, Task 3

Python



Run

```
sns.barplot(  
    ax=axes[0,0],  
    x=avg_by_major.index,  
    y=avg_by_major.values,  
    palette='muted'  
)
```

2. Exercise 7 visualization



Python



Run

Fix these two barplots

Fix 1: Exercise 6 barplot

Replace this:

```
<> Python    
sns.barplot(  
    ax=axes[0,0],  
    x=avg_by_major.index,  
    y=avg_by_major.values,  
    palette='muted'  
)
```

with this:

```
<> Python    
sns.barplot(  
    ax=axes[0, 0],  
    x=avg_by_major.index,  
    y=avg_by_major.values,  
    hue=avg_by_major.index,  
    palette='muted',  
    legend=False  
)
```

Fix 2: Exercise 7 barplot

Replace this:

```
<> Python    
sns.barplot(data=age_stats, x='age_group', y='avg_clv', palette='muted')
```

with this:

```
<> Python    
sns.barplot(  
    data=age_stats,
```

Python

```
... age_group',  
x='age_group',  
y='avg_clv',  
hue='age_group',  
palette='muted',  
legend=False  
)
```



Run

One small improvement for Exercise 4

Your normalization is mathematically correct, but your check prints the **overall** mean and standard deviation instead of checking each column.

You currently have:

Python

```
print("New Mean:", np.round(standardised.mean(), 2))  
print("New Std Dev:", np.round(standardised.std(), 2))
```



Run

Better:

Python

```
print("New Mean:", np.round(standardised.mean(axis=0), 2))  
print("New Std Dev:", np.round(standardised.std(axis=0), 2))
```



Run

This better matches the TODO: "Normalize each column to have mean=0 and std=1."

Other notes

Exercise 4 is done correctly.

Exercise 5 is done correctly. The missing values are filled properly, the grouping is correct, the performance column is created, and the top students logic handles ties reasonably.

Exercise 6 is conceptually done, but the barplot error needs fixing.

Exercise 7 is conceptually done, including the mark  n analysis paragraph, but the barplot error needs fixing there too.

All good now?

I'll check the updated notebook directly and run through the cells enough to confirm whether the TODO fixes are complete and whether any cells still error.

Thought for 36s >

Not fully yet. Your **TODO** logic is **basically done**, but the updated notebook still errors in **Exercise 6, Task 3**.

The remaining problem is this countplot:

```
Python Run
sns.countplot(
    ax=axes[0, 1],
    x='year',
    data=df,
    palette='Blues'
)
```

It gives:

```
Python Run
ValueError: Invalid RGBA argument: None
```

Fix it by adding `hue='year'` and `legend=False`:

```
Python Run
sns.countplot(
    ax=axes[0, 1],
    x='year',
    data=df,
    hue='year',
    palette='Blues',
    legend=False
)
```

Python



Run

```
sns.countplot(
    ax=axes[0, 1],
    x='year',
    data=df,
    hue='year',
    palette='Blues',
    legend=False
)
```

I would also fix these two for the same Seaborn palette issue, even if they only warn now:

Boxplot fix

Python



Run

```
sns.boxplot(
    x='major',
    y='exam_score',
    data=df,
    hue='major',
    palette='pastel',
    legend=False
)
```

Violin plot fix

Python



Run

```
sns.violinplot(
    ax=axes[1, 1],
    x='performance',
    y='exam_score',
    data=df,
    hue='performance',
    palette='Set2',
    order=['Needs Improvement', 'Average', 'Good', 'Excellent'],
    legend=False
)
```



After these changes, you should be good. Your Exercise 4 normalization check is now correct.

Python



Run

So the main remaining issue is not your TODO completion; it is just the Seaborn visualization syntax.



lab0_exercises(2).ipynb
File

Should be fine now

Thought for 40s >

Yes, it looks good now.

I ran the updated notebook from top to bottom, and it executed successfully with **no notebook cell errors**.

Your fixes are in place:

Python



Run

```
standardised.mean(axis=0)  
standardised.std(axis=0)
```

is now correctly checking the normalized columns.

The Seaborn plots that were failing before have also been fixed with `hue=...` and `legend=False`.

Your TODOs are covered for:

Exercise 4: NumPy operations

Exercise 5: Pandas cleaning and analysis

Exercise 6: Visualizations

Exercise 7: CLV integration challenge and analysis paragraph



So yes, this version should be fine to submit.